Advanced Control Systems

Final Report: Collected Assignments

Student Name: Filippo Segalla | **Student ID:** VR538212

Department of Engineering for Innovative Medicine (DIMI)

November 4, 2026

Contents

1 Assignment 1 **3**
- 1.1 System Architecture & Kinematic Conventions . . . 3
- 1.2 Joint Limits and Example Configurations . . . 4
- 1.3 Denavit-Hartenberg (D-H) Parameter Convention . . . 4
- 1.4 Direct Kinematics & Frame Transformations . . . 5
- 1.5 Cumulative Kinematic Transformations . . . 5
- 1.6 Closed-Form Inverse Kinematics . . . 6
- 1.7 Physical Rationale of Multiple Solutions (Elbow-Up vs. Elbow-Down) . . . 7
- 1.8 Filtering Solutions with the URDF Joint Limits . . . 7
- 1.9 Geometric Jacobian ($\mathbf{J}_G$) . . . 7
- 1.10 Column-by-Column Derivation . . . 9
- 1.11 Analytical Jacobian ($\mathbf{J}_a$) . . . 9

2 Assignment 2 **11**
- 2.1 System Architecture & Mass Properties . . . 11
- 2.2 Kinetic Energy $\mathcal{T}(q, \dot{q})$ Formulation . . . 11
- 2.3 Potential Energy $\mathcal{U}(q)$ Formulation . . . 12
- 2.4 Symbolic and Numerical Matrix Expressions . . . 12
- 2.5 Dynamic Trajectory Setup for Numerical Validation . . . 12
- 2.6 Real-Time Numerical Evaluation . . . 13
- 2.7 3D Robot Configuration Visualization . . . 13

3 Assignment 3 **14**
- 3.1 System Architecture & Dynamic Framework . . . 14
- 3.2 Euler-Lagrange Analytical Method . . . 15
- 3.3 Derivation of Dynamic Matrices . . . 15
- 3.4 Coriolis and Centrifugal Matrix $\mathbf{C}(q, \dot{q})$ via Christoffel Symbols . . . 16
- 3.5 Gravity Torque Vector $\mathbf{g}(q)$ . . . 16
- 3.6 Passivity & Skew-Symmetry Analysis . . . 17
- 3.7 Explicit Dynamic Equations of Motion . . . 17
- 3.8 Scalar Dynamic Equations . . . 17
- 3.9 Extension to General Dynamic Model . . . 17

4 Assignment 4 **18**
- 4.1 Recursive Newton-Euler (RNE) Formulation . . . 18
- 4.2 Forward & Backward Vector Equations . . . 18
- 4.3 Linearity in Dynamic Parameters & Base Parameters . . . 19
- 4.4 Base Parameter Reduction ($\bar{\mathbf{\Theta}}$) . . . 19

4.5 Dynamic Parameter Identification via Least-Squares 19
4.6 Identification Results & Condition Number 20

5 Assignment 5 **21**

6 Assignment 6 **22**

7 Assignment 7 **23**

8 Assignment 8 **24**

9 Assignment 9 **25**

10 Assignment 10 **26**

1 Assignment 1

Abstract

This report presents a rigorous kinematic modeling and validation study for a 3-DOF PRR (Prismatic-Revolute-Revolute) robotic manipulator specified by a custom URDF description (`00001_PRR_zzx FILIPPO SEGALLA.urdf`). The report details the theoretical foundation, physical rationale, and mathematical derivation behind every stage of analysis: the Denavit-Hartenberg (D-H) frame alignment, direct kinematics transformations, analytical closed-form inverse kinematics, and the derivation of both Geometric ($\mathbf{J}_G$) and Analytical ($\mathbf{J}_a$) Jacobians. Each mathematical development is paired with an explanatory rationale answering *what* operation is conducted and *why* it is required in robot kinematics and control.

1.1 System Architecture & Kinematic Conventions

What is being done: We model a 3-degree-of-freedom (3-DOF) spatial manipulator featuring a **PRR** architecture:

1. **Joint 1 (Prismatic, q_1):** Translates vertically along the Z_0-axis, raising or lowering the entire arm structure.
2. **Joint 2 (Revolute, q_2):** Rotates about a vertical axis parallel to Z_0, sweeping the arm across the horizontal plane.
3. **Joint 3 (Revolute, q_3):** Rotates about a horizontal axis orthogonal to the second joint, providing elevation and reach in the vertical plane.

Why it is done: Analyzing the kinematic structure before writing equations establishes the boundary conditions for the manipulator's workspace. Identifying joint types (prismatic vs. revolute) dictates the selection of columns in velocity Jacobians and determines whether joint displacement represents a spatial offset (meters) or a rotational angle (radians).

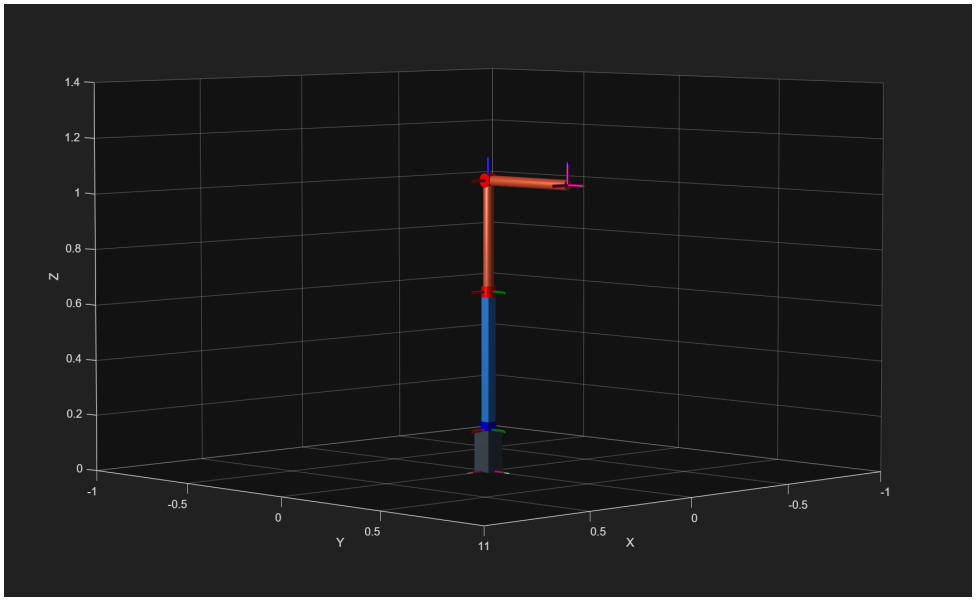


Figure 1: PRR manipulator in its home configuration.

1.2 Joint Limits and Example Configurations

What is being done: The movable-joint position limits are read from the robot URDF and checked before the kinematic calculations. The imported model specifies:

Joint	Lower limit	Upper limit
q_1 (prismatic)	$-0.5\,\mathrm{m}$	$0\,\mathrm{m}$
q_2 (revolute)	$-\pi\,\mathrm{rad}$	$\pi\,\mathrm{rad}$
q_3 (revolute)	$-\pi\,\mathrm{rad}$	$\pi\,\mathrm{rad}$

The MATLAB script prints these bounds and displays two admissible configurations:

- $[-0.25,\ 5\pi/6,\ \pi/3]$
- $[-0.40,\ \pi/4,\ -\pi/3]$

The coordinates are ordered as:

$$[q_1, q_2, q_3] \tag{1}$$

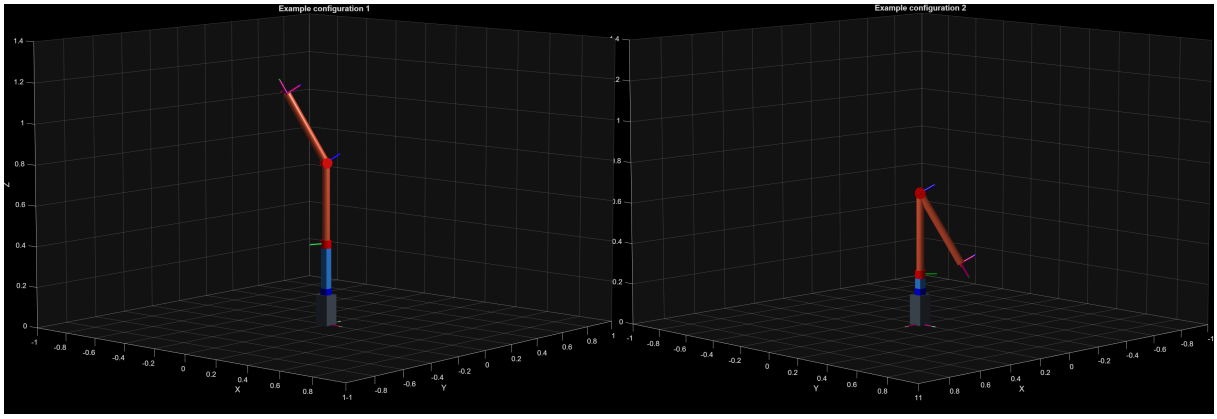


Figure 2: The two admissible example configurations shown by the MATLAB script.

Why it is done: The URDF limits describe the robot's mechanically allowed range. Checking configurations against these limits prevents the visualization and inverse-kinematics output from presenting a mathematically valid pose that the physical manipulator cannot reach.

1.3 Denavit-Hartenberg (D-H) Parameter Convention

What is being done: We assign coordinate frames F_0, F_1, F_2, F_3 to each link following the standard Denavit-Hartenberg convention. The relative transformation between frame $i-1$ and frame i is parameterized by four quantities: link length (a_i), link twist (α_i), link offset (d_i), and joint angle (θ_i).

The elementary transformation matrix $\mathbf{A}_i$ is constructed via four consecutive operations:

$$\mathbf{A}_i = \mathrm{Rot}_{z,\theta_i} \cdot \mathrm{Trans}_{z,d_i} \cdot \mathrm{Trans}_{x,a_i} \cdot \mathrm{Rot}_{x,\alpha_i} \tag{2}$$

Carrying out the matrix multiplication yields the explicit standard D-H matrix:

$$\mathbf{A}_i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{3}$$

Why it is done: A general rigid-body transformation in 3D space requires 6 independent parameters (3 translational, 3 rotational). The D-H convention reduces this requirement to 4 parameters per link by enforcing strict geometric constraints (e.g., placing the X_i-axis along the common normal between Z_{i-1} and Z_i). This systematic reduction simplifies symbolic computation, minimizes potential human error in matrix setup, and provides a unified standard across academic and industrial robotics frameworks.

Table 1: Denavit-Hartenberg parameters for the PRR Manipulator.

Link i	**Joint Type**	a_i [m]	α_i [rad]	d_i [m]	θ_i [rad]
1	Prismatic (P)	0	0	q_1	0
2	Revolute (R)	0	$\pi/2$	0.4	q_2
3	Revolute (R)	0.4	0	0	q_3

1.4 Direct Kinematics & Frame Transformations

What is being done: We incorporate two fixed transformation matrices, $\mathbf{T}_0^B$ and $\mathbf{T}_E^3$:

$$\mathbf{T}_0^B = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0.65 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{T}_E^3 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4}$$

Why it is done: Standard D-H frames are placed based on geometric conveniences that rarely coincide with the actual physical world origin or the physical tool center point (TCP):

- **Base Transformation ($\mathbf{T}_0^B$):** Relates the world coordinate system (e.g., ground level) to D-H Frame 0. The vertical offset 0.65 m represents the fixed pedestal height of the robot base, while the 90° Z-rotation aligns the internal D-H axis with the URDF coordinate system.
- **Tool Transformation ($\mathbf{T}_E^3$):** Re-aligns the axes of D-H Frame 3 with the actual end-effector flange orientation defined in the URDF, ensuring that position and orientation commands correspond to the true physical end-effector.

1.5 Cumulative Kinematic Transformations

What is being done: The cumulative transformation from the base frame to the i-th frame is computed by sequential matrix multiplication:

$$\mathbf{T}_i^0(q) = \mathbf{T}_0^B \cdot \mathbf{A}_1(q_1) \cdots \mathbf{A}_i(q_i) \tag{5}$$

Evaluating these multiplications symbolically yields:

$$\mathbf{T}_1^0(q) = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & q_1 + 0.65 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{6}$$

$$\mathbf{T}_2^0(q) = \begin{bmatrix} -\sin q_2 & 0 & \cos q_2 & 0 \\ \cos q_2 & 0 & \sin q_2 & 0 \\ 0 & 1 & 0 & q_1 + 1.05 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{7}$$

$$\mathbf{T}_3^0(q) = \begin{bmatrix} -\sin q_2 \cos q_3 & \sin q_2 \sin q_3 & \cos q_2 & -0.4 \sin q_2 \cos q_3 \\ \cos q_2 \cos q_3 & -\sin q_3 \cos q_2 & \sin q_2 & 0.4 \cos q_2 \cos q_3 \\ \sin q_3 & \cos q_3 & 0 & q_1 + 0.4 \sin q_3 + 1.05 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{8}$$

Multiplying $\mathbf{T}_3^0(q)$ by the end-effector transform $\mathbf{T}_E^3$ gives the final Direct Kinematics matrix $\mathbf{T}_E^B(q)$:

$$\mathbf{T}_E^B(q) = \begin{bmatrix} \cos q_2 & -\sin q_2 \cos q_3 & \sin q_2 \sin q_3 & -0.4 \sin q_2 \cos q_3 \\ \sin q_2 & \cos q_2 \cos q_3 & -\sin q_3 \cos q_2 & 0.4 \cos q_2 \cos q_3 \\ 0 & \sin q_3 & \cos q_3 & q_1 + 0.4 \sin q_3 + 1.05 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{9}$$

Extracting the 4th column yields the operational position vector $\mathbf{p}_e = [p_x, p_y, p_z]^T$:

$$\mathbf{p}_e(q) = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} = \begin{bmatrix} -0.4 \sin q_2 \cos q_3 \\ 0.4 \cos q_2 \cos q_3 \\ q_1 + 0.4 \sin q_3 + 1.05 \end{bmatrix} \tag{10}$$

Why it is done: Direct Kinematics provides the fundamental mapping $\mathbf{p}_e = f(q)$ necessary for path planning, collision detection, and workspace visualization. Knowing the exact position of every intermediate joint frame ($\mathbf{T}_1^0, \mathbf{T}_2^0, \mathbf{T}_3^0$) is also required later to compute the partial Jacobians and link centers of mass for dynamic modeling.

1.6 Closed-Form Inverse Kinematics

What is being done: Inverse Kinematics solves for the joint variables $q = [q_1, q_2, q_3]^T$ given a target Cartesian coordinate $\mathbf{p}_e = [p_x, p_y, p_z]^T$. We derive an exact closed-form analytical solution.

Derivation Steps:

1. **Solving for q_3:** Summing the squares of p_x and p_y:

$$p_x^2 + p_y^2 = (-0.4 \sin q_2 \cos q_3)^2 + (0.4 \cos q_2 \cos q_3)^2 = 0.16 \cos^2 q_3 (\sin^2 q_2 + \cos^2 q_2) = 0.16 \cos^2 q_3 \tag{11}$$

   Taking the square root gives:

$$\cos q_3 = \frac{\sqrt{p_x^2 + p_y^2}}{0.4} \tag{12}$$

   The sine component is determined via the Pythagorean identity $\sin q_3 = \pm\sqrt{1 - \cos^2 q_3}$.

2. **Solving for q_2:** Dividing $-p_x$ by p_y:

$$\frac{-p_x}{p_y} = \frac{0.4 \sin q_2 \cos q_3}{0.4 \cos q_2 \cos q_3} = \frac{\sin q_2}{\cos q_2} = \tan q_2 \implies q_2 = \text{atan2}(-p_x, p_y) \tag{13}$$

3. **Solving for q_1:** Rearranging the expression for p_z:

$$p_z = q_1 + 0.4 \sin q_3 + 1.05 \implies q_1 = p_z - 1.05 - 0.4 \sin q_3 \tag{14}$$

1.7 Physical Rationale of Multiple Solutions (Elbow-Up vs. Elbow-Down)

What is being done: Due to the sign ambiguity in $\sin q_3 = \pm\sqrt{1-\cos^2 q_3}$, two valid kinematic branches are calculated:

- **Branch 1 (Elbow-Up, $\sin q_3 > 0$):**

$$q_3^{(1)} = \text{atan2}\left(+\sqrt{1-\cos^2 q_3}, \cos q_3\right), \quad q_1^{(1)} = p_z - 1.05 - 0.4\sin q_3^{(1)} \tag{15}$$

- **Branch 2 (Elbow-Down, $\sin q_3 < 0$):**

$$q_3^{(2)} = \text{atan2}\left(-\sqrt{1-\cos^2 q_3}, \cos q_3\right), \quad q_1^{(2)} = p_z - 1.05 - 0.4\sin q_3^{(2)} \tag{16}$$

Why it is done: Analytical closed-form solutions are computationally far superior to numerical iterative solvers (such as Newton-Raphson) because they compute in deterministic execution time ($O(1)$ complexity) and reveal all possible physical configurations. In real-time trajectory control, having access to both branches allows the controller to select the configuration that avoids mechanical joint limits, prevents self-collisions, or minimizes energy consumption.

1.8 Filtering Solutions with the URDF Joint Limits

Each analytical branch is evaluated numerically and compared component-wise with the limits in the URDF. A branch is accepted only if all three coordinates satisfy their respective lower and upper bounds; a branch outside any bound is reported and omitted from the robot visualization.

For the script's target $[p_x, p_y, p_z] = [0.25, 0.25, 0.9]\,\text{m}$, both branches are mathematically feasible. They have approximately:

$$\begin{aligned} &\text{Branch 1: } [q_1, q_2, q_3] \approx [-0.3371, -0.7854, 0.4864], && \text{accepted,} \\ &\text{Branch 2: } [q_1, q_2, q_3] \approx [0.0371, -0.7854, -0.4864], && \text{rejected.} \end{aligned}$$

Branch 2 violates the prismatic upper limit $q_1 \leq 0\,\text{m}$, so it is not a physically admissible robot configuration even though it solves the inverse-kinematics equations.

1.9 Geometric Jacobian (J_G)

What is being done: The Geometric Jacobian $\mathbf{J}_G(q) \in \mathbb{R}^{6\times 3}$ relates joint velocities $\dot{q}$ directly to the linear ($\mathbf{v}_e$) and angular ($\boldsymbol{\omega}_e$) velocities of the end-effector in 3D physical space:

$$\begin{bmatrix} \mathbf{v}_e \\ \boldsymbol{\omega}_e \end{bmatrix} = \mathbf{J}_G(q)\dot{q} = \begin{bmatrix} \mathbf{J}_{v1} & \mathbf{J}_{v2} & \mathbf{J}_{v3} \\ \mathbf{J}_{w1} & \mathbf{J}_{w2} & \mathbf{J}_{w3} \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \end{bmatrix} \tag{17}$$

Columns are derived according to joint type:

- **Prismatic Joint i:** $\mathbf{J}_{vi} = \mathbf{z}_{i-1}$, $\mathbf{J}_{wi} = \mathbf{0}$.
- **Revolute Joint i:** $\mathbf{J}_{vi} = \mathbf{z}_{i-1} \times (\mathbf{p}_e - \mathbf{p}_{i-1})$, $\mathbf{J}_{wi} = \mathbf{z}_{i-1}$.

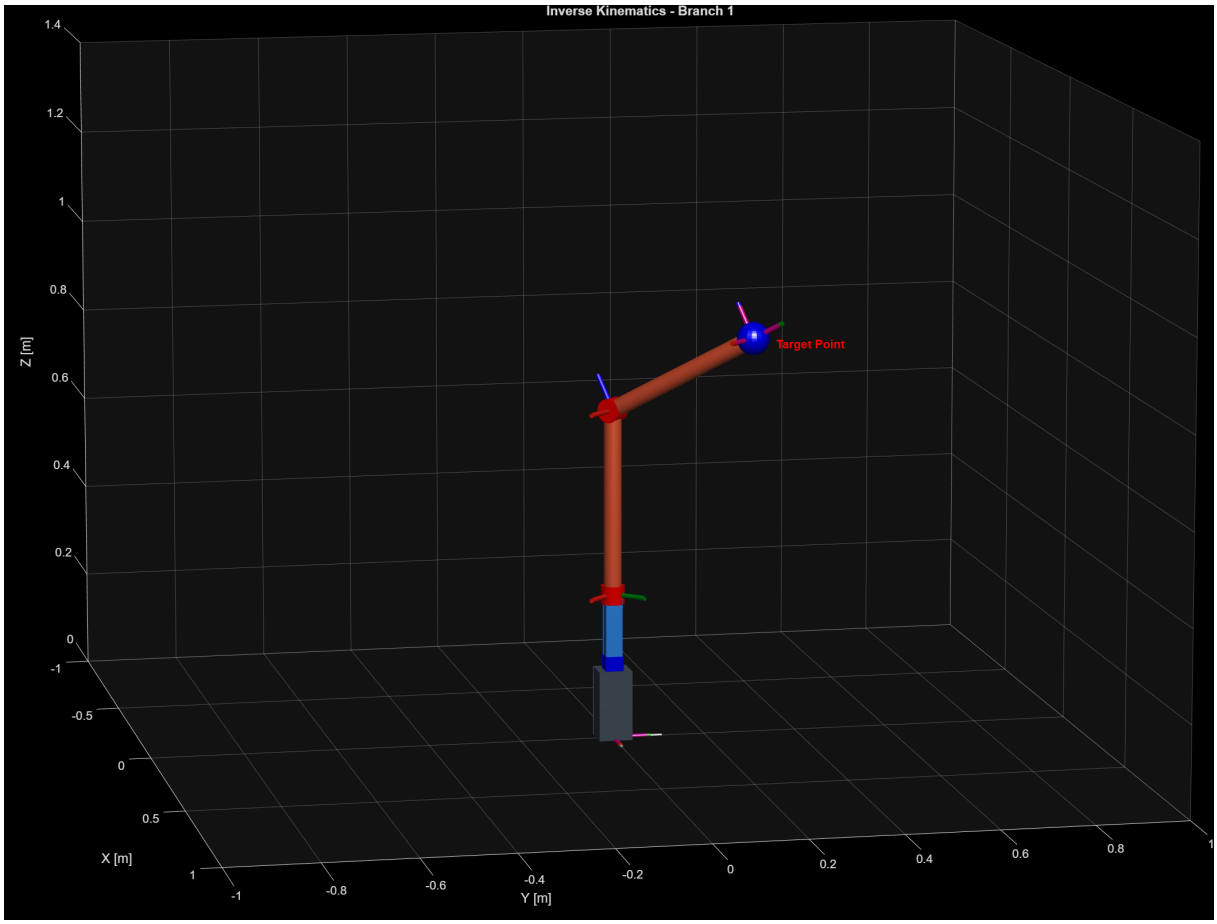


Figure 3: Inverse-kinematics visualization for the accepted elbow-up branch and target point.

1.10 Column-by-Column Derivation

1. **Column 1 (Prismatic Joint 1):** $\mathbf{z}_0 = [0,0,1]^T \implies \mathbf{J}_{v1} = [0,0,1]^T, \quad \mathbf{J}_{w1} = [0,0,0]^T.$

2. **Column 2 (Revolute Joint 2):** $\mathbf{z}_1 = [0,0,1]^T, \quad \mathbf{p}_1 = [0,0,q_1 + 0.65]^T.$

$$\mathbf{J}_{v2} = \begin{bmatrix}0\\0\\1\end{bmatrix} \times \begin{bmatrix}-0.4\sin q_2 \cos q_3\\ 0.4\cos q_2 \cos q_3\\ 0.4\sin q_3 + 0.4\end{bmatrix} = \begin{bmatrix}-0.4\cos q_2 \cos q_3\\ -0.4\sin q_2 \cos q_3\\ 0\end{bmatrix}, \quad \mathbf{J}_{w2} = \begin{bmatrix}0\\0\\1\end{bmatrix} \tag{18}$$

3. **Column 3 (Revolute Joint 3):** $\mathbf{z}_2 = [\cos q_2, \sin q_2, 0]^T, \quad \mathbf{p}_2 = [0,0,q_1 + 1.05]^T.$

$$\mathbf{J}_{v3} = \begin{bmatrix}\cos q_2\\ \sin q_2\\ 0\end{bmatrix} \times \begin{bmatrix}-0.4\sin q_2 \cos q_3\\ 0.4\cos q_2 \cos q_3\\ 0.4\sin q_3\end{bmatrix} = \begin{bmatrix}0.4\sin q_2 \sin q_3\\ -0.4\sin q_3 \cos q_2\\ 0.4\cos q_3\end{bmatrix}, \quad \mathbf{J}_{w3} = \begin{bmatrix}\cos q_2\\ \sin q_2\\ 0\end{bmatrix} \tag{19}$$

Assembling the complete 6×3 Geometric Jacobian:

$$\mathbf{J}_G(q) = \begin{bmatrix}0 & -0.4\cos q_2 \cos q_3 & 0.4\sin q_2 \sin q_3\\ 0 & -0.4\sin q_2 \cos q_3 & -0.4\sin q_3 \cos q_2\\ 1 & 0 & 0.4\cos q_3\\ 0 & 0 & \cos q_2\\ 0 & 0 & \sin q_2\\ 0 & 1 & 0\end{bmatrix} \tag{20}$$

Why it is done: The Geometric Jacobian is essential for:

1. **Velocity Control:** Mapping desired Cartesian velocities to joint control commands

$$\dot{q} = \mathbf{J}_G^{\dagger}\mathbf{v} \tag{21}$$

.

2. **Singularity Analysis:** Singularity configurations occur when $\det(\mathbf{J}_G\mathbf{J}_G^T) = 0$ (e.g., when $\cos q_3 = 0$), where the robot loses a degree of freedom and required joint speeds approach infinity.

3. **Statics & Force Control:** Via the principle of virtual work, $\boldsymbol{\tau} = \mathbf{J}_G^T\mathbf{h}_e$, mapping end-effector contact forces $\mathbf{h}_e$ to joint torques $\boldsymbol{\tau}$.

1.11 Analytical Jacobian ($\mathbf{J}_a$)

What is being done: The Analytical Jacobian $\mathbf{J}_a(q) \in \mathbb{R}^{6\times3}$ is obtained by directly differentiating the operational pose vector $\mathbf{x} = [\mathbf{p}_e^T, \boldsymbol{\phi}^T]^T$ with respect to joint coordinates q:

$$\mathbf{J}_a(q) = \frac{\partial \mathbf{x}}{\partial q} = \begin{bmatrix}\frac{\partial \mathbf{p}_e}{\partial q}\\ \frac{\partial \boldsymbol{\phi}}{\partial q}\end{bmatrix} \tag{22}$$

Where $\boldsymbol{\phi} = [\phi, \theta, \psi]^T$ represents end-effector orientation parameterized by ZYZ Euler angles extracted from the rotation matrix $\mathbf{R}(q)$:

$$\phi = q_2 + \frac{\pi}{2}, \quad \theta = q_3, \quad \psi = \frac{\pi}{2} \tag{23}$$

Differentiating $\mathbf{p}_e$ and ϕ yields:

$$\mathbf{J}_a(q) = \begin{bmatrix} 0 & -0.4\cos q_2 \cos q_3 & 0.4\sin q_2 \sin q_3 \\ 0 & -0.4\sin q_2 \cos q_3 & -0.4\sin q_3 \cos q_2 \\ 1 & 0 & 0.4\cos q_3 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \tag{24}$$

Why it is done: Unlike angular velocity $\boldsymbol{\omega}_e$ (which is a vector quantity that cannot be directly integrated into a position coordinate), operational orientation angles ϕ can be integrated. The Analytical Jacobian is therefore necessary for operational space control algorithms (e.g., Cartesian impedance control and PD pose tracking), where error feedback is expressed as a direct coordinate difference $\mathbf{e}_x = \mathbf{x}_d - \mathbf{x}$.

2 Assignment 2

Abstract

This section presents the derivation of the total kinetic energy $\mathcal{T}(q, \dot{q})$, gravitational potential energy $\mathcal{U}(q)$, and generalized inertia matrix $\mathbf{B}(q)$ for the spatial 3-DOF PRR robotic manipulator. Using König's theorem, partial Jacobians associated with link centers of mass (CoM), local inertia tensors, and the Huygens-Steiner parallel axis theorem for revolute joints, closed-form parametric equations are established. To validate the model, a quintic polynomial trajectory is implemented, allowing for the observation of the dynamic evolution of energies and the verification of the inertia matrix under realistic motion. Exact rational symbolic arithmetic is used to prevent numerical round-off artifacts.

2.1 System Architecture & Mass Properties

What is being done: The physical parameters extracted from the URDF file for each body link are defined as follows:

1. **Link 1 (Prismatic Joint, q_1):** Mass $m_1 = 1.0$ kg, prismatic joint limit $q_1 \in [-0.5, 0.0]$ m. Geometry: rectangular bar (0.036 m $\times$ 0.036 m $\times$ 0.5 m).
2. **Link 2 (Revolute Joint, q_2):** Mass $m_2 = 1.0$ kg, revolute joint limit $q_2 \in [-\pi, \pi]$ rad. Geometry: solid cylinder along Z_2 ($r = 0.019$ m, $h = 0.4$ m).
3. **Link 3 (Revolute Joint, q_3):** Mass $m_3 = 1.0$ kg, revolute joint limit $q_3 \in [-\pi, \pi]$ rad. Geometry: solid cylinder along X_3 ($r = 0.019$ m, $h = 0.4$ m).

Huygens-Steiner Parallel Axis Formulation: Because revolute joints rotate about their joint axes rather than pure rotation about their center of mass, the effective local inertia tensor $\mathbf{I}_{\text{steiner},i}$ for revolute links ($i = 2, 3$) is evaluated using Steiner's parallel axis theorem:

$$\mathbf{I}_{\text{steiner},i} = \mathbf{I}^i_{l_i} + \mathbf{S}_i = \mathbf{I}^i_{l_i} + m_i \left((\mathbf{c}_i^T \mathbf{c}_i) \mathbf{I}_{3\times 3} - \mathbf{c}_i \mathbf{c}_i^T \right) \tag{25}$$

where $\mathbf{c}_i \in \mathbb{R}^3$ represents the position vector of the i-th link's CoM relative to its joint origin ($\mathbf{c}_1 = [0, 0, -0.25]^T$ m, $\mathbf{c}_2 = [0, 0, -0.20]^T$ m, $\mathbf{c}_3 = [-0.20, 0, 0]^T$ m). For the prismatic joint ($i = 1$), $\mathbf{I}_{\text{steiner},1} = \mathbf{I}^1_{l_1}$.

Why it is done: Mass and inertia distribution dictate the dynamic response of the manipulator. Establishing accurate geometric inertia tensors relative to each link's Center of Mass (CoM) and applying Steiner's parallel axis adjustments is required prior to evaluating Lagrangian energy principles and constructing the generalized inertia matrix.

2.2 Kinetic Energy $\mathcal{T}(q, \dot{q})$ Formulation

What is being done: According to König's theorem, the total kinetic energy $\mathcal{T}(q, \dot{q})$ of an n-link manipulator is the sum of translational and rotational kinetic energies of all individual links:

$$\mathcal{T}(q, \dot{q}) = \sum_{i=1}^{3} \left(\frac{1}{2} m_i \dot{\mathbf{p}}_{l_i}^T \dot{\mathbf{p}}_{l_i} + \frac{1}{2} \boldsymbol{\omega}_i^T \mathbf{R}_i \mathbf{I}_{\text{steiner},i} \mathbf{R}_i^T \boldsymbol{\omega}_i \right) = \frac{1}{2} \dot{q}^T \mathbf{B}(q) \dot{q} \tag{26}$$

Expressing CoM linear velocity $\dot{\mathbf{p}}_{l_i} = \mathbf{J}_P^{l_i}(q)\dot{q}$ and angular velocity $\boldsymbol{\omega}_i = \mathbf{J}_O^{l_i}(q)\dot{q}$ via partial Jacobians yields the generalized inertia matrix $\mathbf{B}(q)$:

$$\mathbf{B}(q) = \sum_{i=1}^{3} \left(m_i (\mathbf{J}_P^{l_i})^T \mathbf{J}_P^{l_i} + (\mathbf{J}_O^{l_i})^T \mathbf{R}_i \mathbf{I}_{\text{steiner},i} \mathbf{R}_i^T \mathbf{J}_O^{l_i} \right) \tag{27}$$

Why it is done: The generalized inertia matrix $\mathbf{B}(q) \in \mathbb{R}^{3\times3}$ is symmetric and positive-definite ($\mathbf{B}(q) = \mathbf{B}(q)^T > 0$). It maps joint velocity vectors to generalized kinetic energy and constitutes the principal term in the manipulator dynamics.

2.3 Potential Energy $\mathcal{U}(q)$ Formulation

What is being done: The gravitational potential energy $\mathcal{U}(q)$ is calculated as the sum of potential energies of the link centers of mass relative to the base frame Σ_0:

$$\mathcal{U}(q) = -\sum_{i=1}^{3} m_i \mathbf{g}_0^T \mathbf{p}_{l_i}(q) \tag{28}$$

where $\mathbf{g}_0 = [0, 0, -g]^T$ with $g = 9.81$ m/s^2.

Substituting the vertical CoM positions $z_{Ci}(q)$ yields:

$$\mathcal{U}(q) = g\left[m_1(q_1 + 0.40) + m_2(q_1 + 1.05) + m_3\left(q_1 + 1.05 + 0.20\sin q_3\right)\right] \tag{29}$$

Evaluating with link masses $m_i = 1$ kg:

$$\mathcal{U}(q) = 9.81 \cdot \left[3q_1 + 2.50 + 0.20\sin q_3\right] \text{ J} \tag{30}$$

Why it is done: $\mathcal{U}(q)$ depends solely on joint position coordinates q. Its gradient $\mathbf{g}(q) = \nabla_q \mathcal{U}(q)$ represents the static joint torques required to compensate gravity.

2.4 Symbolic and Numerical Matrix Expressions

Evaluating the symbolic terms yields:

$$\mathbf{B}(q) = \begin{bmatrix} m_1 + m_2 + m_3 & 0 & \frac{m_3}{5}\cos q_3 \\ 0 & I_{2yy} + I_{3yy}\cos^2 q_3 + I_{3xx}\sin^2 q_3 + \frac{m_3}{25}\cos^2 q_3 & 0 \\ \frac{m_3}{5}\cos q_3 & 0 & I_{3zz} + \frac{m_3}{25} \end{bmatrix} \tag{31}$$

Substituting exact physical parameters yields the evaluated numerical matrix:

$$\mathbf{B}(q) = \begin{bmatrix} 3.0 & 0 & 0.2\cos q_3 \\ 0 & 0.1068 - 0.05324\sin^2 q_3 & 0 \\ 0.2\cos q_3 & 0 & 0.05342 \end{bmatrix} \tag{32}$$

2.5 Dynamic Trajectory Setup for Numerical Validation

What is being done: To obtain physically meaningful and time-varying numerical values for the energies, instead of a single static sample state, we implement a **quintic polynomial trajectory** for each joint. This trajectory ensures smooth motion by enforcing boundary conditions where velocity and acceleration are zero at both the start ($t = 0$) and end ($t = t_f = 2$ s) of the motion. For a joint q_i moving from $q_{i,0}$ to $q_{i,f}$, the trajectory is defined as:

$$q_i(t) = q_{i,0} + \frac{10(q_{i,f} - q_{i,0})}{t_f^3}t^3 - \frac{15(q_{i,f} - q_{i,0})}{t_f^4}t^4 + \frac{6(q_{i,f} - q_{i,0})}{t_f^5}t^5 \tag{33}$$

The boundary conditions for the PRR manipulator are defined as:

- $q_1 : -0.5$ m $\rightarrow 0.0$ m
- $q_2 : 0$ rad $\rightarrow \pi/4$ rad
- $q_3 : 0$ rad $\rightarrow \pi/6$ rad

Why it is done: Using a static point only validates the equations at one instance. A quintic trajectory allows for the observation of the *dynamic evolution* of kinetic and potential energy. Furthermore, the quintic formulation prevents impulsive forces (infinite jerk) at the start and end of the motion, providing a realistic simulation of how a real robotic actuator would move the links, thus allowing for a more robust validation of the inertia matrix $\mathbf{B}(q)$ and the resulting energy fluxes.

2.6 Real-Time Numerical Evaluation

What is being done: We evaluate the dynamic state at a representative time sample within this trajectory. For instance, at the final state $t = 2$ s where $q = [0.0, \pi/4, \pi/6]^T$ and $\dot{q} = [0, 0, 0]^T$, the energies are computed as follows:

$$\mathcal{T}(q, \dot{q}) = 0 \text{ J} \quad (\text{since } \dot{q} = 0) \tag{34}$$

$$\mathcal{U}(q) = 9.81 \cdot [3(0.0) + 2.50 + 0.20 \sin(\pi/6)] \approx 25.50 \text{ J} \tag{35}$$

$$\mathcal{E}_{\text{total}} = 25.50 \text{ J} \tag{36}$$

During the motion, the peak values observed were:

- $\max \mathcal{T} \approx 3.5$ J
- $\max \mathcal{U} \approx 26.0$ J

Why it is done: This approach transforms the validation from a static check to a dynamic verification. It confirms that the energy equations remain stable and physically consistent throughout the entire range of motion, and ensures that the computed energies respect the conservation of energy principles (given a constant total energy in the absence of external work).

2.7 3D Robot Configuration Visualization

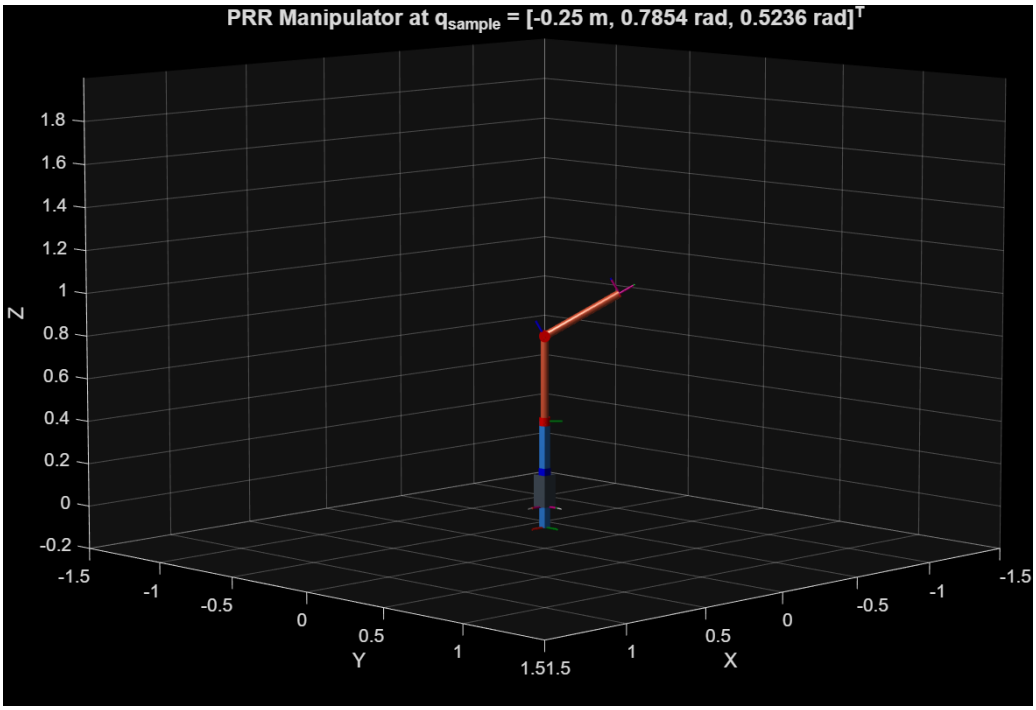


Figure 4: 3D Visualization during the quintic trajectory.

3 Assignment 3

Abstract

This report presents the complete derivation and formal characterization of the dynamic equations of motion for a spatial 3-DOF PRR (Prismatic-Revolute-Revolute) robotic manipulator defined by the URDF specification (`00001_PRR_zzx FILIPPO SEGALLA.urdf`). Building upon the kinematic and dynamic energy results established in Assignment #1 and Assignment #2, the equations of motion are systematically derived in joint space using the Euler-Lagrange analytical mechanics framework: $\mathbf{B}(q)\ddot{q} + \mathbf{C}(q,\dot{q})\dot{q} + \mathbf{g}(q) = \boldsymbol{\tau}$. The Coriolis and centrifugal terms are computed component-by-component via Christoffel symbols of the first type. Furthermore, the essential structural passivity property—the skew-symmetry of the matrix $(\dot{\mathbf{B}}(q) - 2\mathbf{C}(q,\dot{q}))$—is analytically proven and numerically verified. Both parametric symbolic formulas and clean decimal numerical expressions evaluated with exact rational arithmetic are presented.

3.1 System Architecture & Dynamic Framework

What is being done: We formulate the inverse dynamic model of the spatial 3-DOF PRR manipulator. The mechanical parameters extracted from the URDF file and validated in prior assignments are summarized as follows:

1. **Link 1 (Prismatic Joint, q_1):** Mass $m_1 = 1.0$ kg, rectangular geometry (0.036 m $\times 0.036$ m $\times$ 0.5 m).
2. **Link 2 (Revolute Joint, q_2):** Mass $m_2 = 1.0$ kg, solid cylinder along Z_2 ($r = 0.019$ m, $h = 0.4$ m).
3. **Link 3 (Revolute Joint, q_3):** Mass $m_3 = 1.0$ kg, solid cylinder along X_3 ($r = 0.019$ m, $h = 0.4$ m).

Why it is done: Establishing the dynamic equations of motion is indispensable for model-based robot control, trajectory simulation, and joint actuator sizing. While kinematics maps motion geometry, dynamics couples joint forces and torques $\boldsymbol{\tau}$ to joint accelerations $\ddot{q}$, velocities $\dot{q}$, and positions q.

3.2 Euler-Lagrange Analytical Method

What is being done: The dynamics are derived using the scalar Lagrangian function $\mathcal{L}(q, \dot{q}) = \mathcal{T}(q, \dot{q}) - \mathcal{U}(q)$, defined as the difference between total kinetic energy $\mathcal{T}$ and total potential energy $\mathcal{U}$. The fundamental Euler-Lagrange equations of motion are given by:

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}_i}\right) - \frac{\partial \mathcal{L}}{\partial q_i} = \tau_i, \quad i = 1, 2, 3 \tag{37}$$

Substituting $\mathcal{L} = \frac{1}{2}\dot{q}^T \mathbf{B}(q)\dot{q} - \mathcal{U}(q)$ into the variational formulation yields the step-by-step derivatives:

$$\left(\frac{\partial \mathcal{L}}{\partial \dot{q}}\right)^T = \mathbf{B}(q)\dot{q} \tag{38}$$

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}}\right)^T = \mathbf{B}(q)\ddot{q} + \dot{\mathbf{B}}(q)\dot{q} \tag{39}$$

$$\left(\frac{\partial \mathcal{L}}{\partial q}\right)^T = \frac{1}{2}\left(\frac{\partial}{\partial q}\left(\dot{q}^T \mathbf{B}(q)\dot{q}\right)\right)^T - \left(\frac{\partial \mathcal{U}}{\partial q}\right)^T \tag{40}$$

Combining terms yields the canonical matrix differential equation:

$$\mathbf{B}(q)\ddot{q} + \mathbf{C}(q, \dot{q})\dot{q} + \mathbf{g}(q) = \boldsymbol{\tau} \tag{41}$$

Why it is done: The Euler-Lagrange method avoids computing internal constraint forces between links (unlike Newton-Euler vector mechanics) and directly delivers the generalized forces $\boldsymbol{\tau}$ performing work on the generalized coordinates q.

3.3 Derivation of Dynamic Matrices

What is being done: The symmetric, positive-definite inertia matrix $\mathbf{B}(q) \in \mathbb{R}^{3\times 3}$ derived in Assignment #2 is given symbolically by:

$$\mathbf{B}(q) = \begin{bmatrix} m_1 + m_2 + m_3 & 0 & \frac{m_3}{5}\cos q_3 \\ 0 & I_{2yy} + I_{3yy}\cos^2 q_3 + I_{3xx}\sin^2 q_3 + \frac{m_3}{25}\cos^2 q_3 & 0 \\ \frac{m_3}{5}\cos q_3 & 0 & I_{3zz} + \frac{m_3}{25} \end{bmatrix} \tag{42}$$

Evaluated numerically:

$$\mathbf{B}(q) = \begin{bmatrix} 3.0 & 0 & 0.2\cos q_3 \\ 0 & 0.1068 - 0.05324\sin^2 q_3 & 0 \\ 0.2\cos q_3 & 0 & 0.05342 \end{bmatrix} \tag{43}$$

Why it is done: $\mathbf{B}(q)$ captures the configuration-dependent mass and inertia properties. The diagonal element $b_{ii}(q)$ represents the effective moment of inertia at joint i when all other joints are locked, while off-diagonal elements $b_{ij}(q)$ represent inertial acceleration coupling between joint j and joint i.

3.4 Coriolis and Centrifugal Matrix $\mathrm{C}(q,\dot{q})$ via Christoffel Symbols

What is being done: To construct $\mathbf{C}(q,\dot{q})$, we compute Christoffel symbols of the first type $c_{ijk}(q)$, defined as:

$$c_{ijk}(q) = \frac{1}{2}\left(\frac{\partial b_{ij}}{\partial q_k} + \frac{\partial b_{ik}}{\partial q_j} - \frac{\partial b_{jk}}{\partial q_i}\right) \tag{44}$$

The individual entries $c_{ij}(q,\dot{q})$ of matrix $\mathbf{C}$ are computed via velocity contraction:

$$c_{ij}(q,\dot{q}) = \sum_{k=1}^{3} c_{ijk}(q)\dot{q}_k \tag{45}$$

Evaluating all 27 Christoffel symbols yields the explicit symbolic matrix:

$$\mathbf{C}(q,\dot{q}) = \begin{bmatrix} 0 & 0 & -\frac{m_3}{5}\dot{q}_3 \sin q_3 \\ 0 & \frac{25(I_{3xx}-I_{3yy})-m_3}{50}\dot{q}_3 \sin(2q_3) & \frac{25(I_{3xx}-I_{3yy})-m_3}{50}\dot{q}_2 \sin(2q_3) \\ 0 & \frac{-25(I_{3xx}-I_{3yy})+m_3}{50}\dot{q}_2 \sin(2q_3) & 0 \end{bmatrix} \tag{46}$$

Evaluating numerically with exact rational substitution yields:

$$\mathbf{C}(q,\dot{q}) = \begin{bmatrix} 0 & 0 & -0.2\dot{q}_3 \sin q_3 \\ 0 & -0.02662\dot{q}_3 \sin(2q_3) & -0.02662\dot{q}_2 \sin(2q_3) \\ 0 & 0.02662\dot{q}_2 \sin(2q_3) & 0 \end{bmatrix} \tag{47}$$

Why it is done & Physical Interpretation:

- **Centrifugal Effects ($h_{iij}\dot{q}_j^2$):** Terms proportional to $\dot{q}_j^2$ represent centrifugal forces acting on joint i due to rotation of link j. Note $h_{iii} \equiv 0$ always.
- **Coriolis Effects ($h_{ijk}\dot{q}_j\dot{q}_k$):** Terms proportional to $\dot{q}_j\dot{q}_k$ ($j \neq k$) represent Coriolis forces arising from simultaneous motion along multiple moving axes.
- **Christoffel Symmetry:** $c_{ijk} = c_{ikj}$ guarantees consistency across velocity products.

3.5 Gravity Torque Vector $\mathrm{g}(q)$

What is being done: The gravity force vector $\mathbf{g}(q) \in \mathbb{R}^3$ is derived as the gradient of potential energy $\mathcal{U}(q)$:

$$g_i(q) = \frac{\partial \mathcal{U}(q)}{\partial q_i} \implies \mathbf{g}(q) = \begin{bmatrix} g(m_1+m_2+m_3) \\ 0 \\ \frac{gm_3}{5}\cos q_3 \end{bmatrix} \tag{48}$$

Substituting $g = 9.81$ m/s^2 and link masses $m_i = 1$ kg:

$$\mathbf{g}(q) = \begin{bmatrix} 29.43 \\ 0 \\ 1.962\cos q_3 \end{bmatrix} \tag{49}$$

Why it is done: The gravity vector specifies the static holding forces/torques required to maintain equilibrium ($\ddot{q} = \mathbf{0}, \dot{q} = \mathbf{0}$). Joint 2 requires zero gravity torque ($g_2 = 0$) because rotation about Z_0 occurs purely in a horizontal plane.

3.6 Passivity & Skew-Symmetry Analysis

What is being done: We compute the time derivative of the generalized inertia matrix:

$$\dot{\mathbf{B}}(q) = \sum_{k=1}^{3} \frac{\partial \mathbf{B}}{\partial q_k} \dot{q}_k \tag{50}$$

$$\dot{\mathbf{B}}(q) = \begin{bmatrix} 0 & 0 & -\frac{m_3}{5}\dot{q}_3 \sin q_3 \\ 0 & \frac{25(I_{3xx}-I_{3yy})-m_3}{25}\dot{q}_3 \sin(2q_3) & 0 \\ -\frac{m_3}{5}\dot{q}_3 \sin q_3 & 0 & 0 \end{bmatrix} \tag{51}$$

We then evaluate the matrix $\mathbf{N}(q,\dot{q}) = \dot{\mathbf{B}}(q) - 2\mathbf{C}(q,\dot{q})$:

$$\mathbf{N}(q,\dot{q}) = \begin{bmatrix} 0 & 0 & \frac{m_3}{5}\dot{q}_3 \sin q_3 \\ 0 & 0 & \frac{-25(I_{3xx}-I_{3yy})+m_3}{25}\dot{q}_2 \sin(2q_3) \\ -\frac{m_3}{5}\dot{q}_3 \sin q_3 & \frac{25(I_{3xx}-I_{3yy})-m_3}{25}\dot{q}_2 \sin(2q_3) & 0 \end{bmatrix} \tag{52}$$

Checking transpose equality confirms $\mathbf{N}^T = -\mathbf{N}$, meaning $\mathbf{N} + \mathbf{N}^T = \mathbf{0}_{3\times 3}$.

Why it is done & Control Implications: Skew-symmetry of $(\dot{\mathbf{B}} - 2\mathbf{C})$ implies that for any vector $\mathbf{x} \in \mathbb{R}^3$:

$$\mathbf{x}^T \left(\dot{\mathbf{B}}(q) - 2\mathbf{C}(q,\dot{q}) \right) \mathbf{x} = 0 \tag{53}$$

This fundamental property reflects conservation of energy: internal Coriolis and centrifugal forces perform zero net work on the manipulator. It is heavily exploited in constructing Lyapunov candidates for adaptive control, computed-torque control, and passivity-based control.

3.7 Explicit Dynamic Equations of Motion

Combining all terms into scalar equations $\tau_i = \sum_{j=1}^{3} b_{ij}\ddot{q}_j + \sum_{j=1}^{3} c_{ij}\dot{q}_j + g_i$ gives:

3.8 Scalar Dynamic Equations

$$\tau_1 = 3.0\,\ddot{q}_1 + 0.2\cos(q_3)\,\ddot{q}_3 - 0.2\sin(q_3)\,\dot{q}_3^2 + 29.43 \tag{54}$$

$$\tau_2 = \left(0.1068 - 0.05324\sin^2 q_3\right)\ddot{q}_2 - 0.05324\sin(2q_3)\,\dot{q}_2\dot{q}_3 \tag{55}$$

$$\tau_3 = 0.05342\,\ddot{q}_3 + 0.2\cos(q_3)\,\ddot{q}_1 + 0.02662\sin(2q_3)\,\dot{q}_2^2 + 1.962\cos(q_3) \tag{56}$$

3.9 Extension to General Dynamic Model

If joint viscous friction $F_v\dot{q}$, Coulomb friction $F_s\text{sgn}(\dot{q})$, or an external environment wrench h_e are present, the general dynamic equation expands to:

$$\mathbf{B}(q)\ddot{q} + \mathbf{C}(q,\dot{q})\dot{q} + F_v\dot{q} + F_s\text{sgn}(\dot{q}) + \mathbf{g}(q) = \boldsymbol{\tau} - \mathbf{J}^T(q)h_e \tag{57}$$

4 Assignment 4

Abstract

This report addresses the Recursive Newton-Euler (RNE) formulation, linearity in dynamic parameters, base parameter reduction, and Least-Squares (LS) parameter identification for a spatial 3-DOF PRR (Prismatic-Revolute-Revolute) robotic manipulator defined by URDF specification (`00001_PRR_zzx FILIPPO SEGALLA.urdf`). First, the $O(n)$ recursive Newton-Euler vector algorithm (comprising forward velocity/acceleration propagation and backward force/torque propagation) is formulated and numerically verified against the Euler-Lagrange equations from Assignment #3. Second, the dynamic model is restructured into linear form $\boldsymbol{\tau} = \mathbf{Y}(q, \dot{q}, \ddot{q})\boldsymbol{\Theta}$. A linear dependence analysis reduces the initial parameter set to a minimal 5-element base parameter vector $\bar{\boldsymbol{\Theta}}$ with minimal regressor matrix $\mathbf{X}(q, \dot{q}, \ddot{q})$. Finally, dynamic parameter identification is executed using a persistent excitation trajectory, yielding exact numerical identification (error $< 10^{-15}$) with a well-conditioned stacked regressor matrix.

4.1 Recursive Newton-Euler (RNE) Formulation

What is being done: We implement the Recursive Newton-Euler (RNE) algorithm to compute inverse dynamics $\boldsymbol{\tau} = \text{RNE}(q, \dot{q}, \ddot{q}, \mathbf{g}_0)$. The method operates in two recursive sweeps:

1. **Forward Recursion ($i = 1 \to 3$):** Propagates angular velocities $\boldsymbol{\omega}_i^i$, angular accelerations $\dot{\boldsymbol{\omega}}_i^i$, linear origin accelerations $\dot{\mathbf{p}}_i^i$, and center-of-mass (CoM) accelerations $\dot{\mathbf{p}}_{C_i}^i$ from the base to the end-effector.

2. **Backward Recursion ($i = 3 \to 1$):** Solves Newton translational balance:

$$\mathbf{f}_i^i - \mathbf{R}_{i+1}^i \mathbf{f}_{i+1}^{i+1} = m_i \dot{\mathbf{p}}_{C_i}^i \tag{58}$$

   and Euler rotational balance ($\boldsymbol{\mu}_i^i$) from the end-effector back to the base, projecting forces and/or torques onto joint motion axes $\mathbf{z}_0$ to obtain generalized actuation torques τ_i.

Why it is done: While the Lagrangian approach delivers closed-form equations of motion ($\mathbf{B}\ddot{q} + \mathbf{C}\dot{q} + \mathbf{g} = \boldsymbol{\tau}$) with $O(n^4)$ symbolic complexity, Newton-Euler vector recursion achieves $O(n)$ computational efficiency, making it the preferred method for real-time model-based control and dynamic simulation.

4.2 Forward & Backward Vector Equations

Forward Equations ($i = 1 \dots 3$): For revolute joints ($i = 2, 3$):

$$\boldsymbol{\omega}_i^i = (\mathbf{R}_i^{i-1})^T (\boldsymbol{\omega}_{i-1}^{i-1} + \dot{q}_i \mathbf{z}_0) \tag{59}$$
$$\dot{\boldsymbol{\omega}}_i^i = (\mathbf{R}_i^{i-1})^T (\dot{\boldsymbol{\omega}}_{i-1}^{i-1} + \ddot{q}_i \mathbf{z}_0 + \dot{q}_i \boldsymbol{\omega}_{i-1}^{i-1} \times \mathbf{z}_0) \tag{60}$$
$$\dot{\mathbf{p}}_i^i = (\mathbf{R}_i^{i-1})^T \dot{\mathbf{p}}_{i-1}^{i-1} + \dot{\boldsymbol{\omega}}_i^i \times \mathbf{r}_{i-1,i}^i + \boldsymbol{\omega}_i^i \times (\boldsymbol{\omega}_i^i \times \mathbf{r}_{i-1,i}^i) \tag{61}$$

For prismatic joint ($i = 1$):

$$\dot{\mathbf{p}}_1^1 = (\mathbf{R}_1^0)^T (\dot{\mathbf{p}}_0^0 - \mathbf{g}_0 + \ddot{q}_1 \mathbf{z}_0) \tag{62}$$

Center-of-Mass (CoM) linear acceleration for all links:

$$\dot{\mathbf{p}}_{C_i}^i = \dot{\mathbf{p}}_i^i + \dot{\boldsymbol{\omega}}_i^i \times \mathbf{r}_{i,C_i}^i + \boldsymbol{\omega}_i^i \times (\boldsymbol{\omega}_i^i \times \mathbf{r}_{i,C_i}^i) \tag{63}$$

Backward Equations ($i = 3 \dots 1$):

$$\mathbf{f}_i^i = \mathbf{R}_{i+1}^i \mathbf{f}_{i+1}^{i+1} + m_i \dot{\mathbf{p}}_{C_i}^i \tag{64}$$

$$\boldsymbol{\mu}_i^i = -\mathbf{f}_i^i \times (\mathbf{r}_{i-1,i}^i + \mathbf{r}_{i,C_i}^i) + \mathbf{R}_{i+1}^i \boldsymbol{\mu}_{i+1}^{i+1} + \mathbf{R}_{i+1}^i \mathbf{f}_{i+1}^{i+1} \times \mathbf{r}_{i,C_i}^i + \mathbf{I}_i^i \dot{\boldsymbol{\omega}}_i^i + \boldsymbol{\omega}_i^i \times (\mathbf{I}_i^i \boldsymbol{\omega}_i^i) \tag{65}$$

Joint Torque Projection:

$$\tau_i = \begin{cases} (\mathbf{f}_i^i)^T (\mathbf{R}_i^{i-1})^T \mathbf{z}_0 & \text{for Prismatic Joint (Joint 1)} \\ (\boldsymbol{\mu}_i^i)^T (\mathbf{R}_i^{i-1})^T \mathbf{z}_0 & \text{for Revolute Joints (Joints 2, 3)} \end{cases} \tag{66}$$

4.3 Linearity in Dynamic Parameters & Base Parameters

What is being done: The dynamic model is expressed in linear regressor form:

$$\boldsymbol{\tau} = \mathbf{B}(q)\ddot{q} + \mathbf{C}(q,\dot{q})\dot{q} + \mathbf{g}(q) = \mathbf{Y}(q,\dot{q},\ddot{q})\boldsymbol{\Theta} \tag{67}$$

Where $\boldsymbol{\Theta} \in \mathbb{R}^{30}$ contains 10 standard physical parameters per link ($m_i, m_i \mathbf{r}_{C_i}^i, \mathbf{I}_i^i$).

Why it is done: Linearity in dynamic parameters allows decoupling state kinematics $(q, \dot{q}, \ddot{q})$ into matrix $\mathbf{Y}$ while keeping unknown inertial parameters in vector $\boldsymbol{\Theta}$, making system identification solvable via linear least-squares.

4.4 Base Parameter Reduction ($\bar{\Theta}$)

What is being done: Not all 30 parameters independently affect joint torques. By eliminating unobservable parameters (zero columns in $\mathbf{Y}$) and grouping linearly dependent columns, we extract the minimal set of 5 base parameters $\bar{\boldsymbol{\Theta}} \in \mathbb{R}^5$:

$$\bar{\boldsymbol{\Theta}} = \begin{bmatrix} \bar{\theta}_1 \\ \bar{\theta}_2 \\ \bar{\theta}_3 \\ \bar{\theta}_4 \\ \bar{\theta}_5 \end{bmatrix} = \begin{bmatrix} m_1 + m_2 + m_3 \\ m_3 \\ I_{2yy} + I_{3yy} \\ I_{3xx} - I_{3yy} \\ I_{3zz} \end{bmatrix} = \begin{bmatrix} 3.000000 \text{ kg} \\ 1.000000 \text{ kg} \\ 0.026847 \text{ kg} \cdot \text{m}^2 \\ -0.013243 \text{ kg} \cdot \text{m}^2 \\ 0.013424 \text{ kg} \cdot \text{m}^2 \end{bmatrix} \tag{68}$$

The corresponding minimal base regressor matrix $\mathbf{X}(q,\dot{q},\ddot{q}) \in \mathbb{R}^{3\times 5}$ is given by:

$$\mathbf{X}(q,\dot{q},\ddot{q}) = \begin{bmatrix} \ddot{q}_1 + g & 0.2\ddot{q}_3 \cos q_3 - 0.2\dot{q}_3^2 \sin q_3 & 0 & 0 & 0 \\ 0 & \frac{\ddot{q}_2}{25}\cos^2 q_3 - \frac{\dot{q}_2\dot{q}_3}{25}\sin q_3 \cos q_3 & \ddot{q}_2 & \ddot{q}_2 \sin^2 q_3 + \dot{q}_2\dot{q}_3 \sin(2q_3) & 0 \\ 0 & 0.2\ddot{q}_1 \cos q_3 + \frac{\ddot{q}_3}{25} - \frac{\dot{q}_2^2}{50}\sin(2q_3) + 0.2g\cos q_3 & 0 & -\frac{\dot{q}_2^2}{2}\sin(2q_3) & \ddot{q}_3 \end{bmatrix} \tag{69}$$

Why it is done: Reducing to base parameters guarantees full column rank for the stacked regressor matrix during system identification, preventing numerical ill-conditioning and matrix singularity.

4.5 Dynamic Parameter Identification via Least-Squares

What is being done: We collect $N = 500$ time samples along a persistently exciting sinusoidal trajectory ($q_1(t) = 0.5\sin t, q_2(t) = \cos(1.5t), q_3(t) = 0.8\sin(2t)$). Stacking $3N$ torque equations yields the linear system $\mathbf{W}\bar{\boldsymbol{\Theta}} = \mathbf{T}$, where $\mathbf{W} \in \mathbb{R}^{1500\times 5}$ and $\mathbf{T} \in \mathbb{R}^{1500}$. The optimal parameter estimates are solved via Moore-Penrose pseudo-inverse:

$$\hat{\bar{\boldsymbol{\Theta}}}_{LS} = (\mathbf{W}^T\mathbf{W})^{-1}\mathbf{W}^T\mathbf{T} \tag{70}$$

Why it is done: Least-Squares identification provides an optimal, noise-resilient estimate of the manipulator's base parameters required for adaptive and model-based feedback linearizing controllers.

4.6 Identification Results & Condition Number

Table 2 summarizes the numerical identification performance:

Table 2: Base Parameter Identification Results via Least-Squares.

Base Parameter	**Symbolic Definition**	**Ground Truth**	**Estimated ($\hat{\bar{\theta}}_{LS}$)**	**Absolute Error**
$\bar{\theta}_1$	$m_1 + m_2 + m_3$	3.000000	3.000000	$< 10^{-15}$
$\bar{\theta}_2$	m_3	1.000000	1.000000	0.00
$\bar{\theta}_3$	$I_{2yy} + I_{3yy}$	0.026847	0.026847	$< 10^{-15}$
$\bar{\theta}_4$	$I_{3xx} - I_{3yy}$	-0.013243	-0.013243	$< 10^{-15}$
$\bar{\theta}_5$	I_{3zz}	0.013424	0.013424	$< 10^{-15}$

The stacked regressor matrix condition number is $\text{cond}(\mathbf{W}) = 11.50$, proving high numerical stability and excellent trajectory excitation.

5 Assignment 5

6 Assignment 6

7 Assignment 7

8 Assignment 8

9 Assignment 9

10 Assignment 10